STELLA AI (STLA Guardian AI) — Complete Project Knowledge Base

1. PROJECT OVERVIEW

Project Name: Stella AI (STLA Guardian AI) **Domain:** Automotive IoT, AI-Powered Emergency Dispatch **Team Members:** Adithya Naik, Achal Sangami, Bindu

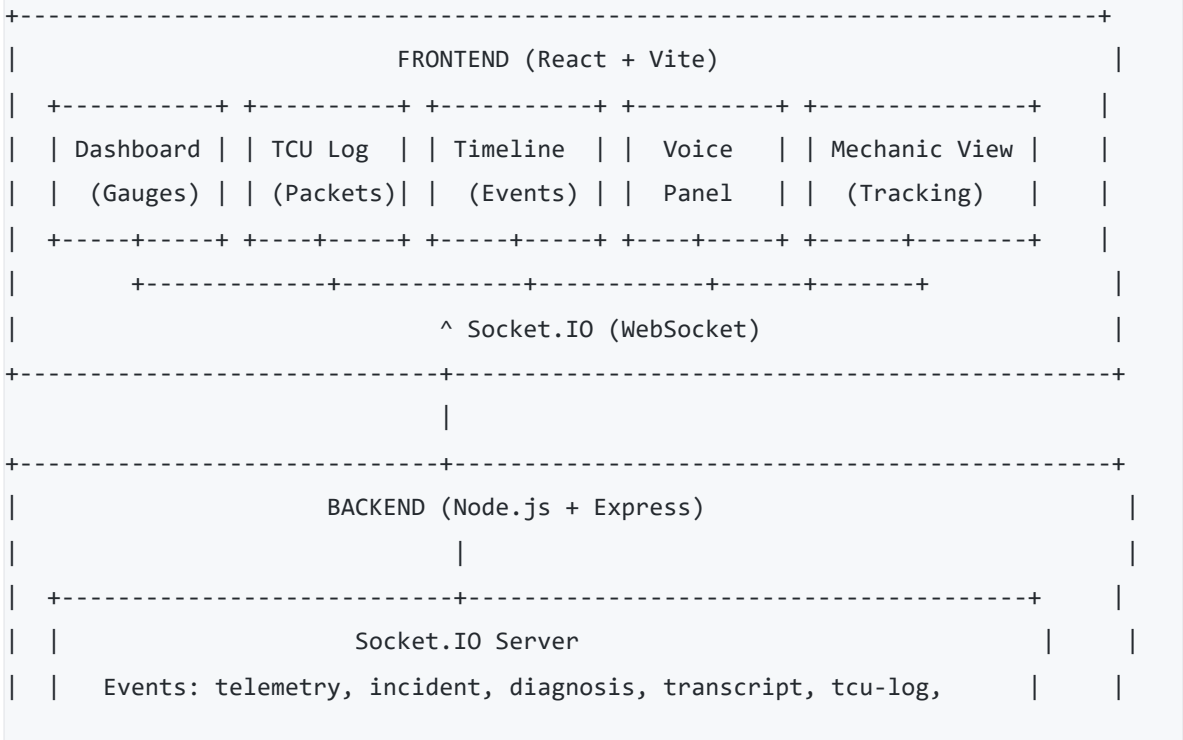
Stella AI is an end-to-end, fully automated vehicle diagnostics and emergency dispatch system built for the Stellantis STLA platform. When a vehicle experiences a fault, the system captures real-time Telematics Control Unit (TCU) data, uses an AI (Large Language Model) to instantly diagnose the root cause, and automatically triggers a sequential 3-call voice pipeline to consult an engineer, dispatch a service provider, and notify the stranded driver — all without any human call-center intervention. The driver receives a WhatsApp message with a live GPS tracking link to monitor the rescue vehicle's arrival.

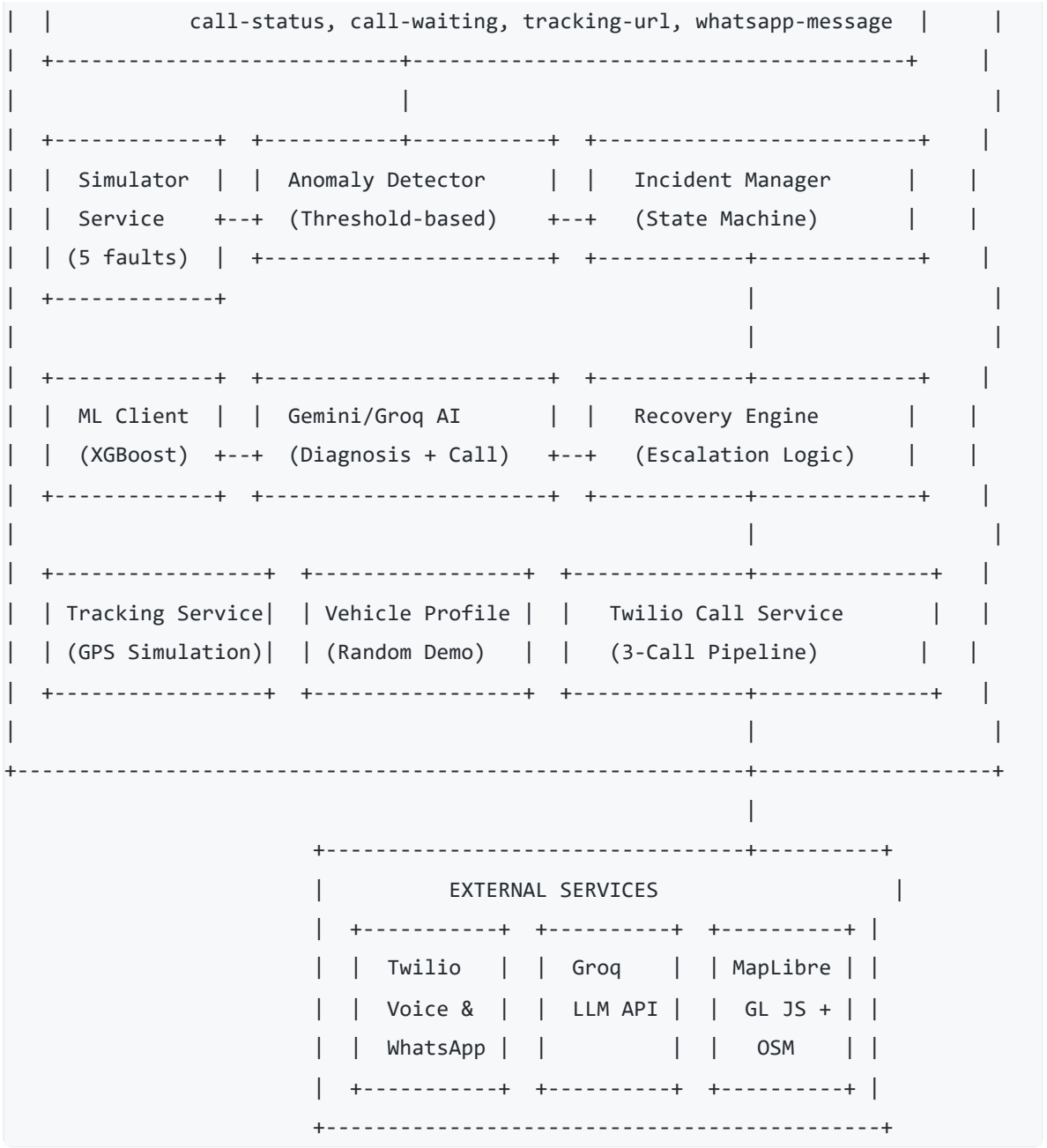
---

2. SYSTEM ARCHITECTURE

2.1 High-Level Architecture Diagram

```
plaintext
```





2.2 Data Flow (Step by Step)

```plaintext
1. User clicks "Simulate Engine Fault" on Dashboard
       |
2. SimulatorService generates degrading telemetry every 1.5 seconds
       |
3. AnomalyDetector checks each reading against thresholds
       | (warning / critical)
4. MLClient sends data to XGBoost model for prediction (optional)
       |
5. GeminiDiagnosis (Groq LLM) analyzes telemetry -> produces rootCause,
   severity (1-5), confidence (0-100), and recommendations
```

```
       |
6. IncidentManager creates an Incident with unique ID and tracks state
       |
7. RecoveryEngine triggers -> waits 2 seconds -> calls TwilioCallService
       |
8. CALL 1: Engineer (two-way voice) -> AI presents diagnosis, asks
   "Should we send a mechanic or a tow truck?"
       |
9. CALL 2: Mechanic/Tow (two-way voice) -> AI asks "Are you available?"
   If "No" -> retries up to 3 times. If "Yes" -> sends GPS + tracking link.
       |
10. CALL 3: Driver (one-way notification) -> AI says "Don't panic,
    a [mechanic/tow truck] is on the way. ETA: 12 minutes."
       |
11. WhatsApp messages sent to both driver and service provider
       |
12. TrackingService creates a live GPS session -> driver sees a map
    with the rescue vehicle moving toward them in real time
       |
13. "Driver Has Been Helped!" popup when service provider arrives
```

---

3. TECH STACK (Complete)

3.1 Backend

Technology	Version	Purpose
Node.js	v24.x	Server runtime
Express.js	4.21	REST API and webhook handling
Socket.IO	4.8	Real-time bidirectional WebSocket events
Twilio SDK	5.4	Programmable Voice calls + WhatsApp messaging
Groq SDK	0.5	Ultra-fast LLM inference (LLaMA 3.3 / GPT-OSS 20B)
@google/generative-ai	0.24	Gemini 1.5 Flash (fallback AI provider)
uuid	11.1	Unique incident and session IDs
dotenv	16.4	Environment variable management
cors	2.8	Cross-origin request handling

3.2 Frontend

Technology	Version	Purpose
React	18.3	Component-based UI framework
Vite	5.4	Lightning-fast dev server and build tool
Tailwind CSS	3.4	Utility-first CSS framework
Framer Motion	11.18	Smooth animations and transitions
Zustand	5.0	Lightweight global state management
Socket.IO Client	4.8	Real-time data from backend
Lucide React	0.469	Icon library
React Router DOM	7.1	Client-side routing

3.3 External Services

Service	Purpose
Groq Cloud API	Sub-second LLM inference for diagnosis + voice responses
Twilio Programmable Voice	Automated phone calls with speech recognition
Twilio WhatsApp Sandbox	Automated WhatsApp messages with tracking links
MapLibre GL JS	Open-source map rendering (no API key needed)
OpenStreetMap	Free map tile provider
Ngrok	Local development tunnel for Twilio webhooks
Render	Cloud deployment platform

3.4 ML/AI Layer (Adithya's Component)

Technology	Purpose
XGBoost (Python)	Edge Analytics: primary fault classification from sensor patterns
FastAPI (Python)	ML model serving endpoint at localhost:8000
Groq LLaMA 3.3 70B	Layer 2 Contextual AI: validates ML prediction, generates root cause

---

4. BACKEND SERVICES (Detailed)

4.1 SimulatorService (simulator.js)

- Generates realistic vehicle telemetry with 25+ parameters
- Supports 5 fault types: engine, electrical, coolant, transmission, brake
- Each fault has a severity curve that worsens over time (tick-based)
- Parameters include: RPM, oil pressure, engine temp, voltage, current, battery SOC, coolant temp/pressure/flow rate, vibration, brake temp/pressure, ABS error count, transmission temp/pressure, gear slip, CAN error count, GPS coordinates

4.2 AnomalyDetector (anomalyDetector.js)

- Threshold-based anomaly detection with 16 monitored parameters
- Each parameter has 4 thresholds: warning-low, warning-high, critical-low, critical-high
- Returns anomaly objects with: parameter name, value, unit, severity level, descriptive message

4.3 MLClient (mlClient.js)

- HTTP client that sends telemetry history to XGBoost model at localhost:8000/predict
- 1200ms timeout with AbortController
- Returns null gracefully if ML service is unavailable (fault-tolerant)

4.4 GeminiDiagnosis (geminiDiagnosis.js)

- Two AI providers supported: Groq (primary), Google Gemini (fallback)
- diagnoseFromTelemetry(): Takes last 10 readings + ML prediction -> returns structured JSON diagnosis with rootCause, severity, confidence scores, risk assessment, and recommendations
- generateCallResponse(): Powers the two-way phone conversation. Takes engineer's speech, diagnosis context, and conversation history -> returns a concise phone-appropriate AI reply (max 45 words)
- extractResolutionIntent(): Analyzes transcript to determine if the engineer wants to auto-fix or escalate
- Fallback system: 5 hardcoded diagnosis maps (coolant, electrical, engine, transmission, brake) used when AI APIs are unavailable
- Weather integration: Can answer live weather questions during the call

4.5 IncidentManager (incidentManager.js)

- Central state machine for the entire incident lifecycle
- States: detected -> diagnosing -> diagnosed -> calling -> in_call -> calling_mechanic -> calling_driver -> escalated/resolved
- Manages: transcript lines, TCU logs, timeline events, call phase/status, tracking info
- Every state change is broadcast to all connected frontends via Socket.IO

4.6 RecoveryEngine (recoveryEngine.js)

- Decision layer between diagnosis and action
- All faults escalate to the engineer call chain
- Includes scenario guard: prevents old demo timers from interfering with new demos
- If Twilio is not configured, falls back to simulated call flow

4.7 TwilioCallService (twilioCall.js) — 877 lines, the core engine

- Call 1 (Engineer): Two-way voice call using TwiML + Gather. AI speaks diagnosis, listens to engineer's response via speech recognition, and generates a dynamic AI reply
- Call 2 (Mechanic/Tow): Two-way call asking availability. If "yes" -> shares vehicle location + tracking URL. If "no" -> retries up to 3 times
- Call 3 (Driver): One-way notification call with reassuring message about help being dispatched
- WhatsApp integration: Sends GPS location to service provider + tracking link to driver

- Simulated flow: Complete mirror of the live Twilio flow for demos without credentials
- Transition guards: Prevents duplicate calls via engineerTransitionQueued, mechanicTransitionQueued, driverCompleted flags
- Conversation history: Maintains full conversation array for context-aware AI responses

4.8 TrackingService (tracking.js)

- Creates unique tracking sessions with service provider starting 3-5km away
- Uses linear interpolation to simulate movement toward the driver
- Provides real-time position, ETA countdown, progress percentage, and arrival detection
- Generates realistic Karnataka vehicle registration numbers

4.9 VehicleProfile (vehicleProfile.js)

- Generates randomized demo vehicles from the Stellantis brand family
- Includes: brand, model, trim, model year, registration, color, VIN, curb weight, dimensions
- Used by the AI to provide accurate vehicle-specific answers during calls

---

5. FRONTEND COMPONENTS (Detailed)

5.1 App.jsx

- Main application shell with 6 tabs: Dashboard, TCU Log, Timeline, Voice Panel, Mechanics, System Logs
- Establishes the Socket.IO connection (auto-detects localhost vs production URL)
- Routes all socket events to Zustand store

5.2 Dashboard.jsx

- Real-time vehicle health gauges (RPM, Temperature, Battery, Oil Pressure)
- Health score calculation (0-100%)
- Fault simulation buttons (Engine, Electrical, Coolant, Transmission, Brake)
- Vehicle system map highlighting affected components

5.3 TcuLog.jsx

- Visualizes raw TCU network packets in real-time
- Shows: packet ID, source ECU, payload data, transmission status
- Protocol info bar (CAN-FD, 4G-LTE, gateway, interval)
- Auto-scrolling with animated entry

5.4 VoicePanel.jsx

- Live call transcript viewer with speaker labels (AI, Engineer, Mechanic, Driver)
- 3-call pipeline progress indicator with visual states
- Call waiting animation between phases
- Filter by call phase

5.5 MechanicView.jsx

- Service provider dispatch status
- Live tracking card with "Open Live Map" button
- WhatsApp delivery confirmation
- Google Maps link to vehicle location

5.6 Timeline.jsx

- Chronological event timeline of the entire incident lifecycle
- Color-coded events: red (fault), blue (diagnosis), yellow (calling), green (resolved)

5.7 SystemLogs.jsx

- Raw system log viewer for debugging
- Categories: call, error, resolution, system

5.8 useStore.js (Zustand Store)

- Global state: telemetry, incident, diagnosis, transcript, tcuLogs, trackingUrl, callWaiting, healthScore, systemLogs
- API calls: simulateFault(), resetSystem()
- Dynamic API_BASE: uses localhost:3001 in dev, empty string (same origin) in production

---

6. API ENDPOINTS

REST Endpoints

Method	Route	Purpose
POST	/simulate-fault	Trigger a fault simulation (body: { type: "engine" })
POST	/reset	Reset all systems to nominal
GET	/health	System health check
GET	/track/:sessionId	Serve live tracking HTML page
GET	/api/tracking/:sessionId/position	JSON position data for map

Twilio Webhook Endpoints

Method	Route	Purpose
POST	/twilio/voice	Initial TwiML for engineer call
POST	/twilio/gather	Process engineer's speech response
POST	/twilio/engineer-status	Call status callback
POST	/twilio/mechanic-voice	TwiML for mechanic/tow call
POST	/twilio/mechanic-gather	Process mechanic's speech response
POST	/twilio/mechanic-status	Mechanic call status callback
POST	/twilio/driver-voice	TwiML for driver notification call
POST	/twilio/driver-status	Driver call status callback

---

7. ENVIRONMENT VARIABLES

Variable	Example	Purpose
GROQ_API_KEY	gsk_...	Groq LLM API key for AI inference
TWILIO_ACCOUNT_SID	AC62f7ca...	Twilio account identifier
TWILIO_AUTH_TOKEN	d3e8052b...	Twilio authentication
TWILIO_FROM_NUMBER	+14846006731	Twilio phone number (outgoing)
TWILIO_TO_NUMBER	+919535728467	Default call recipient
TWILIO_MECHANIC_NUMBER	+919535728467	Mechanic/tow recipient
TWILIO_DRIVER_NUMBER	+919535728467	Driver recipient
TWILIO_WHATSAPP_NUMBER	+14155238886	Twilio WhatsApp sandbox number
TWILIO_WEBHOOK_BASE_URL	https://xxx.ngrok-free.dev	Public URL for Twilio callbacks
PORT	3001	Server port

---

8. DEPLOYMENT

Local Development

```plaintext
Terminal 1: ngrok http 3001
Terminal 2: cd backend && npm run dev
Terminal 3: cd frontend && npm run dev
```

Production (Render)

- render.yaml blueprint included in the repository
- Build command: cd frontend && npm install && npm run build && cd ../backend && npm install
- Start command: cd backend && node server.js
- Backend serves the built React app from ../frontend/dist in production
- No ngrok needed — Render provides a public URL automatically

---

9. JUDGE Q&A — GENERAL QUESTIONS (For Any Teammate)

Q: What is the main problem your project solves? A: Traditional roadside assistance relies on slow manual call centers. Drivers wait on hold, operators lack real-time vehicle data, and wrong resources get dispatched. Stella AI eliminates all of this by using AI to instantly diagnose the fault from live telemetry and autonomously manage the entire dispatch process through automated phone calls.

Q: How does the system work end-to-end? A: When a vehicle fault is detected, the TCU streams telemetry data to our backend. An anomaly detector flags critical readings, which are then sent to an XGBoost model for pattern classification and to a Large Language Model for root-cause analysis. Based on the diagnosis, the system automatically makes 3 sequential phone calls: first to an engineer for approval, then to a mechanic or tow truck for availability, and finally to the driver to inform them. The driver also receives a WhatsApp message with a live tracking link.

Q: Why did you choose Groq over OpenAI or Gemini? A: Speed. Groq processes around 500 tokens per second compared to OpenAI's ~50 tokens per second. In a live phone call, the AI needs to respond in under a second or the person on the other end hears dead silence. Groq's sub-second inference makes the conversation feel natural and fluid. It also has a generous free tier.

Q: What happens if the AI API goes down? A: We built a full fallback system. If Groq is unavailable, it falls back to Google Gemini. If both are down, we have hardcoded rule-based diagnosis maps for all 5 fault types that provide accurate diagnoses without any AI API. The system never stops working.

Q: What happens if Twilio is not configured? A: The entire call flow runs in a simulated mode. The TwilioCallService has a complete simulateCall() method that mirrors the real flow with realistic delays and transcript generation. This allows full demos without Twilio credentials.

Q: Is the telemetry data real? A: Currently, the telemetry is generated by our SimulatorService, which produces realistic data mimicking a real CAN-FD network. The simulator models 25+ vehicle parameters and progressively degrades them over time to simulate a genuine fault. The architecture is designed so that swapping in real OBD-II hardware data requires only replacing the simulator module.

Q: How do you handle multiple concurrent faults? A: Each simulation resets the previous state. The system uses a scenarioSequence token to ensure that delayed callbacks from an old fault don't interfere with a new simulation. The IncidentManager tracks exactly one active incident at a time.

Q: What is the role of WebSockets in the project? A: Socket.IO provides real-time, bidirectional communication between the backend and all connected frontends. It streams live telemetry readings (every 1.5 seconds), AI diagnosis results, call status updates, voice transcripts, TCU packet logs, and tracking URLs instantly to the dashboard without polling.

Q: Can you deploy this without a local machine running? A: Yes. We have a render.yaml blueprint that deploys both the frontend and backend as a single Render Web Service. In production, the Node.js server serves the built React app, and Twilio webhooks point directly to the Render URL. No ngrok or local machine is needed.

Q: What is the tracking page built with? A: The tracking page uses MapLibre GL JS with OpenStreetMap tiles. It shows a car icon (driver, static) and a truck icon (service provider, moving). The backend's TrackingService uses linear interpolation to simulate the provider moving toward the driver. The page polls for position updates every 2 seconds. When the provider arrives, a "Driver Has Been Helped!" popup appears.

---

10. JUDGE Q&A — ADITHYA NAIK (System Architecture & ML Model)

Q: Explain the system architecture you designed. A: The system follows a layered, event-driven architecture. The data layer consists of the Simulator and Anomaly Detector, which generate and validate telemetry. The intelligence layer has two components: an XGBoost ML model for pattern-based fault classification and a contextual LLM (Groq) for root-cause analysis and conversational AI. The orchestration layer uses the IncidentManager as a state machine and the RecoveryEngine as the decision router. The communication layer is the TwilioCallService that manages the 3-call pipeline. Everything is connected via Socket.IO for real-time event broadcasting.

Q: Why did you choose XGBoost for the ML model? A: XGBoost is ideal for tabular sensor data classification. It handles the 25+ numerical features (RPM, voltage, temperature, pressure) efficiently, provides fast inference at the edge, and gives interpretable feature importances that help us understand which sensors contributed most to a prediction. It also performs well with relatively small training datasets.

Q: What does the ML model actually predict? A: The XGBoost model takes a window of recent telemetry readings and outputs a fault classification (engine, electrical, coolant, transmission, brake, or normal), a confidence score (0-100%), a severity level (1-5), and the most affected parameters. This primary prediction is then sent to the LLM for contextual validation and enrichment.

Q: Why do you have two AI layers instead of just one? A: This is a deliberate architecture decision. The XGBoost model acts as a fast, reliable "edge" classifier that works purely on numerical patterns — it's deterministic and predictable. The LLM acts as a "contextual" layer that validates the ML prediction, adds natural-language explanations, assesses real-world risk, and generates human-readable advice. This two-layer approach gives us both speed and intelligence.

Q: How does the anomaly detection work? A: The AnomalyDetector uses a threshold-based approach. Each of the 16 monitored parameters has 4 thresholds: warning-low, warning-high,

critical-low, and critical-high. For example, engine temperature has a warning range of 70-105 degrees C and a critical range of 50-120 degrees C. If a reading exceeds any threshold, an anomaly is raised with a severity level and descriptive message.

Q: How does the simulator produce realistic fault patterns? A: Each fault type has a unique degradation function. For example, the engine fault increases RPM using a sinusoidal oscillation (simulating misfires), drops oil pressure linearly, raises engine temperature, and increases vibration. A severity multiplier Math.min(tick/8, 1) ensures the fault worsens gradually over 8 ticks, mimicking how real faults escalate. Random noise is added for realism.

Q: What training data did you use for the XGBoost model? A: We generated synthetic training data using the simulator's fault functions across all 5 fault types plus normal operation. Each sample is a vector of the 25+ sensor parameters at a given severity level. The model was trained on thousands of samples with balanced class representation.

Q: How does the ML service communicate with the backend? A: The ML model is served via FastAPI at localhost:8000. The Node.js backend sends POST requests to /predict with the telemetry history as JSON. The response contains the classification, confidence, and severity. If the ML service is unavailable (timeout after 1200ms), the system gracefully degrades to the LLM-only diagnosis path.

Q: Can this architecture work with real vehicle hardware? A: Yes. The simulator is a pluggable module. To connect real hardware, you would replace the SimulatorService with an OBD-II reader library that reads CAN bus frames from the vehicle's OBD-II port. The rest of the pipeline — anomaly detection, ML prediction, AI diagnosis, and call dispatch — remains unchanged.

Q: How do you ensure the system is fault-tolerant? A: At every layer we have fallbacks. If the ML service is down, the diagnosis still works via the LLM. If Groq is down, Gemini is used. If both are down, hardcoded rule-based diagnoses kick in. If Twilio is not configured, the entire call flow runs in simulation mode. The system is designed to never crash or stop responding, regardless of which external service fails.

---

11. JUDGE Q&A — ACHAL SANGAMI (Backend, Calling Features & Integration)

Q: Explain the 3-call pipeline in detail. A: After the AI diagnoses the fault, the RecoveryEngine waits 2 seconds and calls initiateEngineerCall(). This creates a Twilio call with a webhook URL pointing to /twilio/voice. When the engineer picks up, Twilio sends a POST request to that URL, and our server responds with TwiML that says the diagnosis and uses Gather with speech recognition. When the engineer speaks, Twilio POSTs the transcript to /twilio/gather, where we send it to the LLM for an AI response. After the engineer hangs up, the status callback triggers handleEngineerCallStatus(), which waits 2 seconds and initiates the mechanic call. The same pattern repeats for calls 2 and 3.

Q: How does Twilio speech recognition work in your project? A: We use Twilio's Gather verb with input="speech" and speechTimeout="2". When the person on the call speaks, Twilio converts their speech to text using its built-in speech-to-text engine and sends the transcription as a POST parameter (SpeechResult) to our gather webhook URL. We then pass this text to the Groq LLM for a contextual AI response, convert that response back to speech using Twilio's Say verb, and continue the conversation.

Q: What is TwiML? A: TwiML (Twilio Markup Language) is an XML-based language that tells Twilio what to do during a phone call. We use Say to make the AI speak, Gather to listen for the human's response with speech recognition, Pause for natural conversation pauses, and Hangup to end the call. Our backend dynamically generates TwiML in response to Twilio's webhook requests.

Q: What is ngrok and why is it needed? A: Ngrok creates a public HTTPS tunnel to your local machine. When Twilio makes a call, it needs to send webhook HTTP requests (speech results, status updates) back to our server. But our local machine at localhost:3001 isn't accessible from the internet. Ngrok gives us a URL like https://abc123.ngrok-free.dev that forwards requests to localhost:3001. In production on Render, ngrok is not needed because Render provides a public URL.

Q: How do you handle the engineer's decision? A: In the /twilio/gather endpoint, we analyze the engineer's speech. If it contains keywords like "mechanic", "send someone", or "dispatch", we set engineerDecision = 'mechanic'. If it contains "tow", "towing", or "truck", we set engineerDecision = 'tow truck'. We also send the speech to the LLM, which returns a context-aware response. If no clear decision is captured by the time the call ends, we default to dispatching a tow truck as the safest option.

Q: What happens if the mechanic says no? A: The handleMechanicGather() method checks the mechanic's speech. If it contains negative keywords (no, can't, unavailable, busy), the AI responds with "Okay, I will contact the next available [role]." The call ends, and the handleMechanicCallStatus() callback retries by calling initiateMechanicCall() again. This retries up to 3 times. If all 3 attempts fail, the system defaults to notifying the driver directly.

Q: How does the WhatsApp integration work? A: We use the Twilio WhatsApp Sandbox. After the mechanic confirms availability, _sendTrackingWhatsApp() sends two messages: one to the service provider with the vehicle's Google Maps location, vehicle number, and driver's contact; and one to the driver with the root cause, service provider details, ETA, and a live tracking link. Messages are sent from whatsapp:+14155238886 (Twilio Sandbox) to whatsapp:+91XXXXXXXXXX.

Q: How does Socket.IO work in the backend? A: Socket.IO is initialized on the same HTTP server as Express. Every state change in the system emits a Socket.IO event. For example, when new telemetry is generated, we emit io.emit('telemetry', reading). When the AI adds a transcript line, we emit io.emit('transcript', line). The frontend listens for these events and updates the Zustand store in real-time. This means every connected browser instantly sees changes without polling.

Q: How did you make the project deployable to Render? A: Three changes: (1) Frontend API URLs use import.meta.env.DEV to auto-detect localhost vs production. (2) The backend checks for process.env.RENDER_EXTERNAL_URL to set the Twilio webhook base URL instead of ngrok. (3) In production, the Express server serves the built React app from ../frontend/dist as static files with a catch-all route for SPA routing. The render.yaml blueprint automates the build and start commands.

Q: What was the hardest technical challenge? A: Preventing duplicate calls. Twilio sends multiple status callbacks (ringing, in-progress, completed) for each call, and JavaScript's setTimeout can trigger overlapping flows. I solved this with transition guard flags (engineerTransitionQueued, mechanicTransitionQueued, driverCompleted) that ensure each phase only transitions once. Without these, the system would make 2-3 calls to the mechanic simultaneously.

---

12. JUDGE Q&A — BINDU (Frontend & Testing)

Q: How is the frontend structured? A: The frontend is a single-page application built with React and Vite. The main App.jsx component has a tab-based navigation with 6 views: Dashboard, TCU Log, Timeline, Voice Panel, Mechanics, and System Logs. All state is managed globally using Zustand, and real-time updates come through a single Socket.IO connection established in App.jsx.

Q: How does state management work? A: We use Zustand, a lightweight state management library. The useStore.js file defines a single store with all the application state: telemetry, incident, diagnosis, transcript, tcuLogs, trackingUrl, healthScore, and more. Each Socket.IO event listener in App.jsx calls the corresponding Zustand action (like updateTelemetry() or addTranscriptLine()), which triggers a re-render only in the components that subscribe to that specific piece of state.

Q: Why did you choose Zustand over Redux? A: Zustand is much simpler than Redux. It requires no boilerplate — no action types, no reducers, no dispatch. You just define state and actions in a single function call. For a real-time application with many rapid state updates (telemetry every 1.5 seconds, transcripts, call statuses), Zustand's minimal overhead and selective re-rendering make it more performant and easier to maintain.

Q: How do the real-time gauges on the Dashboard work? A: The Dashboard subscribes to the telemetry state in the Zustand store. Every 1.5 seconds, the backend emits a new telemetry reading via Socket.IO, which updates the store. The gauge components re-render with the new values. The gauges use SVG arcs or CSS transforms to visually represent values like RPM, temperature, and battery SOC. The health score is calculated dynamically from all parameter readings.

Q: How does the Voice Panel show live transcripts? A: The Voice Panel subscribes to the transcript array in the Zustand store. Each time the backend emits a transcript event (when the

AI speaks or when speech is recognized), a new line is pushed to the array. The panel renders each line with the speaker label, text, and a call-phase tag. Framer Motion adds a smooth slide-in animation for each new line. There's also a filter to view transcripts by call phase (engineer, mechanic, driver).

Q: What animations did you use and why? A: We use Framer Motion for all animations. Key animations include: (1) Page transitions with fade and slide effects when switching tabs, (2) Transcript lines sliding in from the bottom with opacity fade, (3) Call pipeline progress indicators transitioning between states, (4) TCU log entries animating in with a staggered delay. These animations make the dashboard feel alive and responsive, which is critical for a monitoring application.

Q: How did you test the application? A: Testing was done at multiple levels: (1) Component-level testing by verifying each UI component renders correctly with various states (no data, loading, error, populated). (2) Integration testing by running the full stack locally and simulating all 5 fault types. (3) Edge-case testing for scenarios like rapid button clicking, no Twilio credentials, API timeouts, and mechanic declining multiple times. (4) Responsiveness testing across different screen sizes.

Q: What bugs did you find and fix? A: Key bugs included: (1) Duplicate transcript entries when Twilio sent multiple gather callbacks — fixed with the emittedTranscriptKeys Set that tracks unique message keys. (2) Race condition where the engineer's status callback would fire after the mechanic call already started, corrupting state — fixed with call-phase guards that ignore callbacks from previous phases. (3) Auto-scroll on the TCU log not working after many entries — fixed with a useRef and scrollIntoView in a useEffect. (4) The health score flickering during normal-to-fault transition — fixed with debounced state updates.

Q: How does the TCU Log page work? A: The TCU Log page displays raw network packets as they arrive from the backend. Each packet shows: a unique packet ID (e.g., TCU-000142), the source ECU (Engine, Transmission, Battery, Thermal, Body), key payload values, and a status indicator. The protocol info bar at the top shows CAN-FD as the protocol, 4G-LTE as the network, and the sampling interval. Packets auto-scroll with the newest at the bottom, and each entry has a slide-in animation.

Q: How does the frontend know if it's running locally or in production? A: Vite provides import.meta.env.DEV, which is true during npm run dev and false after npm run build. We use this to set the Socket.IO URL and API base URL. In development, it connects to http://localhost:3001. In production (after building and deploying to Render), it uses window.location.origin, which means all requests go to the same server that served the page.